uPMU AFE en LTspice — resultados, correcciones y pendientes

Todo lo de aquí es simulado, no medido. Lo único medido que se usa son los datos del firmware en `STM32/uPMU/docs/RESULTADOS.md` (reloj del ADC, tiempo de muestreo, ruido del ADC), que se citan como tales.

- **LTspice 26.0.2** (`C:\Program Files\ADI\LTspice\LTspice.exe`), modo batch `-b -ascii`.
- **Python** como contraste independiente: modelo cerrado exacto (`afe_model.py`) y cadena completa (`simulate_chain.py`).

Reproducir: `run_ltspice.bat` (todo, ~5 min) o `python codigo/run_ltspice.py` (solo LTspice; regenera los netlists y el esquemático por su cuenta).

---

1. Resumen

	Antes	Después
Fase V−I a 60 Hz	**0,0000°** (exacto por construcción)	**−0,1444°**
Fase V−I a 50 Hz	0,0000°	−0,1733°
TVE p95 calibrado, f0 ± 5 Hz (Colombia)	0,009 / 0,015 %	**0,151 / 0,183 %**
TVE p95 calibrado, f0 ± 5 Hz (Chile)	0,011 / 0,014 %	**0,219 / 0,268 %**
TVE p95 calibrado en f0	0,007 / 0,013 %	0,008 / 0,013 % (igual)

El cambio viene de **modelar la corriente de magnetización de los transformadores**. Antes eran ideales y el error de fase del sensor se metía como una constante aleatoria (±0,3° en V, ±1° en I), que la calibración de un punto borra entera. El error real **depende de la frecuencia** (ε ∝ 1/f), así que calibrar en f0 **no** lo quita fuera de f0.

Sigue muy por debajo del 1 % de la norma clase M. Roza el objetivo interno de 0,2 % en Chile a f0−5 Hz (0,219 %, máx. 0,229 %). La corrección propuesta en §6 lo baja a 0,059 %.

---

2. LTspice contra el modelo cerrado de Python

Dos caminos independientes: LTspice resuelve el circuito numéricamente, `afe_model.py` lo resuelve en forma cerrada (escalera completa: CT con magnetizante, RC, Rs y la parásita del pin, con vcm como masa de alterna).

Umbrales del enunciado: 0,1 % en módulo, 0,05° en fase.

Respuesta en frecuencia (un canal excitado a la vez)

Canal	f [Hz]	LTspice |H|	Python |H|	Fase LTspice	Fase Python
V	50	7,579415e−3 V/V	7,579446e−3	+0,1471°	+0,1474°
V	60	7,579421e−3 V/V	7,579453e−3	−0,1266°	−0,1261°
I	50	1,499661e−2 V/A	1,499668e−2	+0,3204°	+0,3207°
I	60	1,499685e−2 V/A	1,499691e−2	+0,0179°	+0,0183°

Fase V−I: LTspice −0,1733° (50 Hz) y −0,1444° (60 Hz); el modelo cerrado da los mismos valores a las cuatro cifras.

Por bandas, la mayor diferencia entre los dos:

Banda	Δmódulo	Δfase
10–100 Hz	0,0004 %	0,0008°
100 Hz–1 kHz	0,0030 %	0,0081°
1–5 kHz	0,0285 %	0,0225°
5–100 kHz	0,0511 %	**0,1780°** (a 100 kHz)

Transitorio (recorrido en el pin del ADC)

Comparado **muestra a muestra en los instantes en que muestrea el ADC**, que es más exigente que comparar picos y es lo que de verdad ve el firmware:

Red	Carga	V(adcv) / V(adci) [V]	Δ contra el modelo cerrado
Colombia	nominal	0,364–2,936 / 0,908–2,392	0,011 / 0,009 mV
Colombia	V 120 %, I 200 %	0,106–3,194 / 0,165–3,135	0,014 / 0,016 mV
Chile	nominal	0,411–2,889 / 0,908–2,392	0,011 / 0,008 mV
Chile	V 120 %, I 200 %	0,163–3,137 / 0,165–3,135	0,014 / 0,014 mV

0,016 mV = **0,020 LSB**.

La única diferencia por encima del umbral

Dónde	Diferencia	Origen
5 kHz–100 kHz	hasta 0,178° a 100 kHz (0,051 % en módulo)	El modelo cerrado no incluye la **capacidad de juntura de los 1N4148** (Cjo 4 pF, ≈2,5 pF con 1,65 V de inversa, dos diodos por nodo). Suma ≈5 pF al pin y mueve su polo de 23 MHz a 13 MHz. A 5 kHz da 0,022°, a 60 Hz da 0,0003°: **irrelevante en la banda de trabajo**.

Por debajo de 5 kHz todo queda dentro de umbral: ≤0,029 % en módulo y ≤0,023° en fase.

---

3. Compatibilidad del netlist con LTspice

Lo que se revisó y lo que resultó:

Punto	Resultado
`.options reltol=1e-6 vntol=1e-9 abstol=1e-12`	LTspice acepta las tres y siguen haciendo falta: con el `reltol` de fábrica el solver deja ~2 mV (2,5 LSB).
`.param`, `SIN(...)`, `PULSE(...)`	Sin cambios.
`.tran`	**Corregido.** Se emite `.tran 2u <stop> <start> 2u`: el `Tstart` hace que LTspice simule el arranque pero guarde solo la ventana en régimen (ver §4.5).
`.save`	Reduce el `.raw` a los nodos que interesan; sin esto los transitorios de 5 s ocupan cientos de MB.
`.meas`	Sintaxis de LTspice (`FIND mag(V(x)) AT 60`). Se leen en la GUI y en el `.log`. La extracción de verdad se hace leyendo el `.raw` desde Python, que no depende de la sintaxis de `.meas`.
`.step` + `mc()`	Para el Monte Carlo (`uPMU_AFE_mc_*.cir`): `.step param run 1 200 1` con `mc(valor, tol)`. `mc()` vuelve a sortear en cada corrida y es determinista por índice, así que Colombia y Chile usan las mismas tiradas y la comparación entre redes es pareada.
Modelo del 1N4148	**Corregido**, ver §4.3.
Exportar datos	`.raw` ASCII (`-b -ascii`) leído con `ltspice_raw.py`. El parser honra el campo `Offset:`, que LTspice usa cuando `.tran` lleva `Tstart`; sin eso el eje de tiempo sale corrido.
Separar los `.step` al leer	Con un solo punto de frecuencia no hay forma de cortar las 200 corridas dentro del `.raw`. Por eso los netlists de Monte Carlo usan `.ac list 50 60`: el salto de 60 a 50 marca el corte (y de paso mide en las dos redes).

---

4. Correcciones aplicadas

Todas son correcciones de **modelo o de sintaxis**. Ningún valor de componente cambió (lo que cambiaría el diseño está en §6, solo propuesto).

4.1 `.ac` excitaba los dos canales a la vez — y con 2000 A

Antes: `Vmains ... AC 1` y `Ict ... AC 1` activas simultáneamente. `AC 1` en `Ict` son 1 A en el **secundario**, es decir **2000 A primarios**. Ese nivel absurdo acoplaba al canal V a través de las capacidades parásitas a masa.

Medido: el canal V salía con **0,39 % de error en módulo y 0,012° en fase** (7,550e−3 contra 7,580e−3 a 60 Hz). La verificación anterior lo describía como «viene de estimular el CT con 1 A a la vez» y lo daba por bueno; era un error del netlist, no un efecto físico.

Después: un canal por netlist (`uPMU_AFE_ac_V.cir`, `uPMU_AFE_ac_I.cir`) y con la unidad física correcta: la fuente del canal I inyecta `1/2000` A en el secundario, de modo que `V(adci)` sale directamente en **V por amperio primario**, comparable con la `Hi()` de `simulate_chain.py`.

Para la fase V−I y el Monte Carlo se usa un netlist aparte con **las dos fuentes a su nivel nominal** (120 V / 35 A), donde el acoplamiento por el nodo VCM sí es el real: queda en 5e−5 %, despreciable.

4.2 Los transformadores eran ideales (la corrección importante)

Antes: `Fv vcm bv Vsp 1`, una fuente de corriente ideal. Sin error de fase del sensor. En `simulate_chain.py` el desfase se metía aparte como constante aleatoria (±0,3° / ±1°), que la calibración de un punto borra por completo.

Después: modelo referido al secundario con magnetizante `Lm` en paralelo y devanado `Rw` en serie. El adelanto vale ε = (Rw + Rb)/(ω·Lm), **proporcional a 1/f**.

`Lm` no se inventó: se **deriva del error de fase de la hoja de datos**.

	ZMPT101B	SCT-013
Dato de partida	«phase angle error ≤ 20'» con 2 mA y **100 Ω** de muestreo	sin dato en la hoja: se conserva el supuesto de 1° con 30 Ω
Rw supuesto	150 Ω `CONFIRMAR`	20 Ω `CONFIRMAR`
Lm derivada	**136,8 H**	**9,12 H**
ε a 50 Hz con la carga real	+0,827°	+1,000°
ε a 60 Hz con la carga real	+0,689°	+0,833°

Con la carga real (470 Ω) el ZMPT101B da **0,69–0,83°**, no los 0,3° que se suponían: el ensayo de la hoja es con 100 Ω, y este diseño usa **4,7 veces** esa carga (ver §6.1).

Resultado en fase, a 60 Hz: el RC atrasa −0,814° y el CT adelanta +0,689° (V) / +0,833° (I), así que casi se cancelan por canal, pero **no entre canales**:

f	Fase V (ideal → real)	Fase I (ideal → real)	Fase V−I (ideal → real)
50 Hz	−0,6796 → **+0,1471°**	−0,6796 → **+0,3204°**	0,0000 → **−0,1733°**
60 Hz	−0,8155 → **−0,1266°**	−0,8155 → **+0,0179°**	0,0000 → **−0,1444°**

Lo que la calibración en f0 **no** quita, porque depende de la frecuencia. (El firmware ya compensa el polo RC en `Hv_nom` / `Hi_nom`, así que lo que sobrevive es solo el término del transformador, ε(f) = ε(f0)·f0/f.)

	Residuo en fase V−I	Residuo en fase absoluta, canal V	…canal I
Colombia, f0 − 5 Hz / f0 + 5 Hz	−0,0131° / +0,0111°	+0,0626° / −0,0530°	+0,0757° / −0,0641°
Chile, f0 − 5 Hz / f0 + 5 Hz	−0,0193° / +0,0158°	+0,0919° / −0,0752°	+0,1112° / −0,0909°

La fase **absoluta** se degrada casi seis veces más que la diferencia V−I, porque los dos canales se mueven en el mismo sentido y se cancelan bastante al restarlos. Para un PMU, que publica el fasor contra UTC, la que importa es la absoluta: ±0,11° en Chile son 1,9 mrad, es decir **0,19 % de TVE solo por este término**.

Lo mismo se llevó a `simulate_chain.py`: el desfase constante se reemplazó por el modelo físico, con ±30 % de tolerancia en `Lm`.

4.3 El modelo del 1N4148 estaba hecho a mano

Antes: `.model DCL D(Is=2.5n N=1.75 Rs=0.6 Cjo=4p)` — cuatro parámetros, sin ruptura inversa, sin `Vj` / `M` (así que `Cjo` no se ajustaba con la polarización), sin corriente de recombinación.

Después: el modelo estándar (Philips/NXP), el mismo que distribuye LTspice, con `Bv=110`, `Isr/Nr`, `Vj=.5`, `M=.3333`, `Tt=11.54n`. Es lo que permite dar por bueno el cálculo de fuga en operación normal (§5.3) y el de limitación en sobretensión.

4.4 Faltaba la red de muestreo del ADC

Antes: solo `Cpv adcv 0 5e-12`, sin interruptor ni condensador de muestreo: no se podía comprobar el asentamiento.

Después: interruptor `SW` con `Ron = R_ADC`, condensador de muestreo `C_ADC_SH` y una ventana de muestreo con la temporización **medida en el firmware** (`STM32/uPMU/docs/RESULTADOS.md`, T3.1/T3.2): reloj del ADC = AHB/4 = 15 MHz y tiempo de muestreo = 61,5 ciclos → **4,10 µs**. Resultado en §5.4.

También se subió la parásita del pin de 5 pF a **7 pF**, que es lo que dice la nota del diagrama de conexión típica de la hoja del STM32F303 (pad + PCB).

4.5 Los transitorios no llegaban al régimen permanente

Antes: `.tran 0 100m`. Con el CT ideal daba igual. Con el magnetizante no: aparecen **dos transitorios lentos** que a 100 ms están lejísimos de acabarse.

1. La continua del magnetizante: τ = Lm/(Rw+Rb) = **221 ms** (V) y **182 ms** (I).
2. La **resonancia de Lm con los 10 µF de VCM**: 4,3 Hz en el canal V y 16,7 Hz en el canal I, que decae con τ = 2L/R = **0,44 s** y **0,37 s**.

Con 100 ms de simulación el pico del canal I en sobrecarga salía **3,640 V** (por encima de 3,3 V) cuando el valor en régimen es 3,135 V: un falso positivo de saturación.

Después: `.tran 2u 5.1 5 2u` — se simulan 5 s y se guardan los últimos 100 ms. Comprobado por dos caminos: con 2 s de asentamiento quedaban 1,7 mV (2 LSB) de offset en el canal I a 50 Hz; con 5 s bajan a 0,008 mV; y al quitar `Cvcm1` el resto desaparece también, lo que confirma que el modo es el LC.

Esto no es solo numérico. Al energizar, el front-end real tarda **unos segundos** en asentarse y en ese rato tiene un tono de 4–17 Hz encima de la señal. El estimador ajusta un término de continua, así que la parte lenta la absorbe, pero el tono de 16,7 Hz del canal I no es continua. **Recomendación: descartar los primeros ~3 s de reportes después de encender** (`CONFIRMAR` con la placa).

4.6 Otras

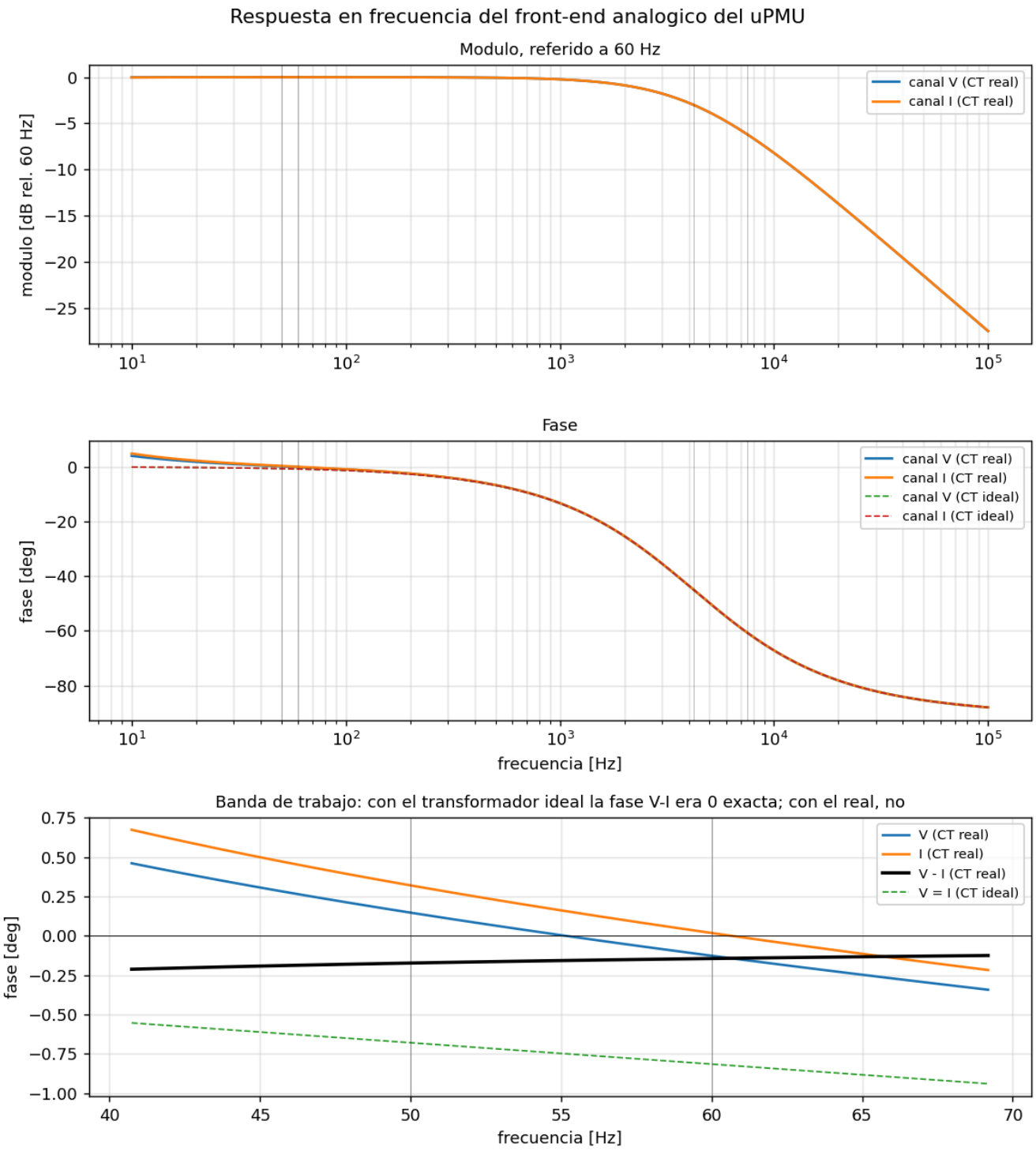
- `.ac dec 40 1 200k` → **`.ac dec 100 10 100k`**, el rango que pide el enunciado y con más resolución.
- `.meas` añadidas (módulo y fase a 50/60 Hz; máximos del transitorio; corrientes de diodo).
- Monte Carlo añadido: `.step param run 1 200 1` con `mc(valor, tol)`, R al 1 % y C al 5 %.
- `ltspice_raw.py`: lector de `.raw` ASCII con autocomprobación (`python codigo/ltspice_raw.py` compara un RC contra su fórmula).
- `afe_model.py`: modelo cerrado **exacto** (escalera completa con CT, RC, Rs y la parásita del pin) con autocomprobación. Antes la referencia era la fórmula de un polo, que es la que hacía parecer «correctos» los 0,8° de error.
- `R_PRI_ZMPT`: parámetro nuevo para la resistencia del primario, que se suma a Rp. Se deja en **0** a propósito: solo reduce la señal, así que 0 es lo conservador para el margen contra 3,3 V. `CONFIRMAR`: medirla.

---

5. Resultados

5.1 Respuesta en frecuencia

- Ganancia: canal V 7,5794e−3 V/V, canal I 1,49969e−2 V/A primario (ambas a 60 Hz).
- fc = 4,22 kHz, igual en los dos canales (Rbv+Rfv = Rbi+Rfi = 3770 Ω). Se verificó: la parte RC de la fase es idéntica en los dos canales a 5 decimales.
- Atenuación respecto a 60 Hz: −0,2 dB a 1 kHz, −3,0 dB a 4,22 kHz, −6,2 dB a 7,5 kHz, −17,1 dB a 30 kHz, −27,5 dB a 100 kHz.
- Con el CIC3 (R=4) en cascada: **−17,3 dB a 7,5 kHz**, −36,9 dB a 11 kHz, −48,4 dB a 22 kHz. **El peor rechazo de todo el rango 7,5–60 kHz son esos −17,3 dB a 7,5 kHz**, justo en el Nyquist de la salida de 15 kS/s. 15 kHz y 30 kHz no son el peor caso: son ceros exactos del CIC.

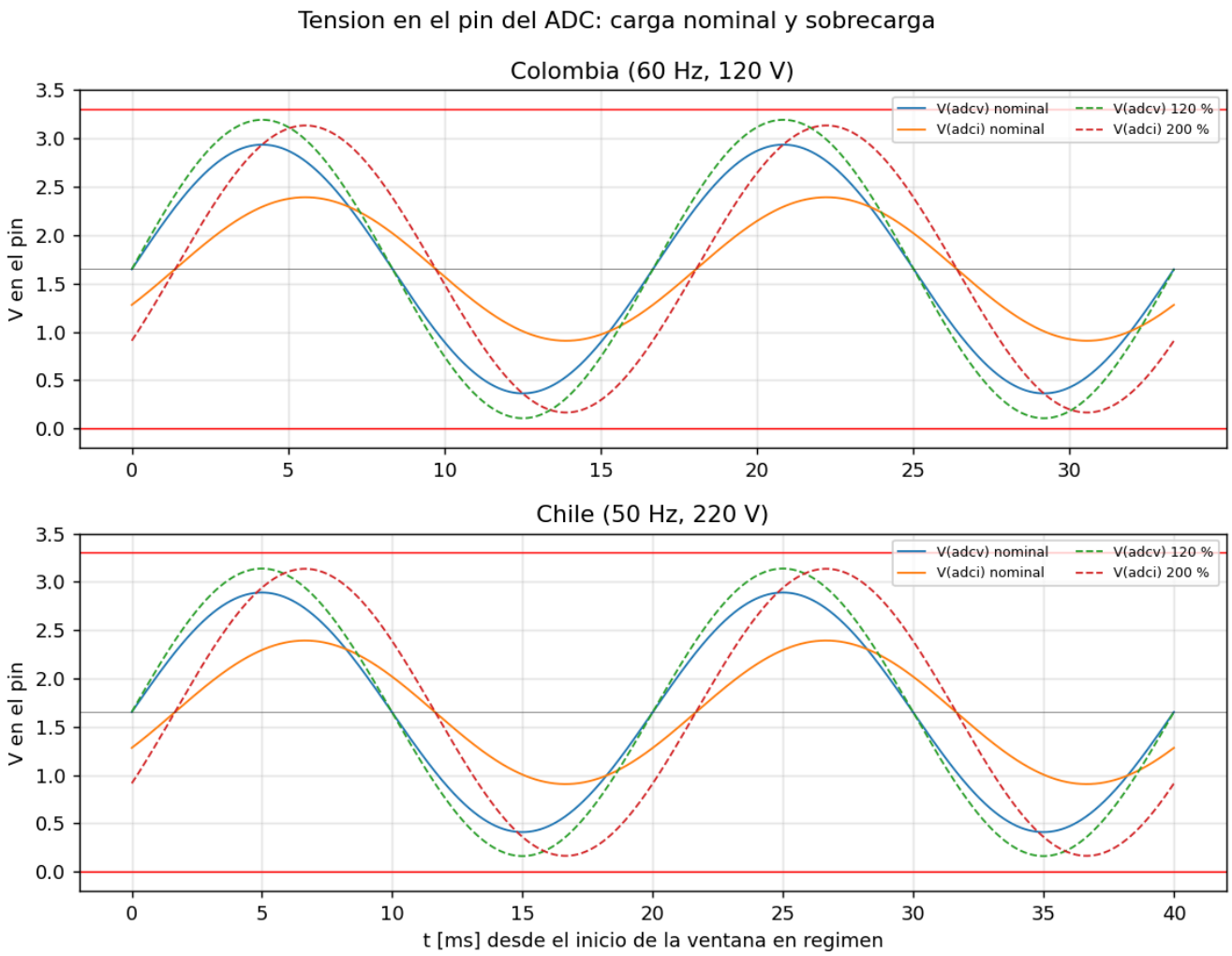


El tercer panel es el que muestra el hallazgo: con el transformador ideal (línea punteada) la fase V−I era cero exacta en toda la banda.

5.2 Transitorio y margen

Red	Carga	V(adcv)	V(adci)	Margen a 0 V	Margen a 3,3 V
Colombia	nominal	0,364–2,936 V	0,908–2,392 V	0,364 V	0,364 V
Colombia	V 120 %, I 200 %	0,106–3,194 V	0,165–3,135 V	**0,106 V**	**0,106 V**
Chile	nominal	0,411–2,889 V	0,908–2,392 V	0,411 V	0,411 V
Chile	V 120 %, I 200 %	0,163–3,137 V	0,165–3,135 V	**0,163 V**	**0,163 V**

Cabe, pero el caso peor deja **106 mV (132 LSB, 3,2 % del fondo de escala)**. Es poco margen para absorber tolerancias de VDDA, offset de VCM y el transitorio de encendido de §4.5. Ver la propuesta 6.2.



5.3 Clamps

- **Operación normal (incluida la sobrecarga de 120 %/200 %):** corriente máxima en los cuatro 1N4148 = **7,8 nA**, que es la fuga inversa del modelo. No conducen. No distorsionan.
- **Sobretensión (V 400 %, I 600 %):** conducen y limitan el pin a **−0,58 V / +3,88 V**, con 0,45–0,62 mA por diodo.

Aviso: 3,88 V son VDDA + 0,58 V, por encima del máximo absoluto de VDDA + 0,3 V de la hoja del STM32. Quien protege de verdad es **Rs = 1 kΩ**, que mantiene la corriente inyectada (≈0,6 mA) muy por debajo del límite de ±5 mA por pin. El 1N4148 no puede hacer más: su caída directa a esas corrientes es de 0,6–0,7 V. `CONFIRMAR` contra la tabla de máximos absolutos del F303 si se quiere acotar mejor; si hace falta, un BAT54 (0,3 V) en vez del 1N4148 sí entraría en el VDDA + 0,3 V (propuesta, no aplicada).

5.4 Muestreo del ADC: ¿asientan 12 bits?

Sí, con mucho margen.

- τ = (Rs + R_ADC)·C_SH = (1000 + 1000)·5 pF = **10,0 ns**.
- 12 bits piden $\ln(2^{13})\cdot\tau$ = **90 ns**.
- Hay **4,10 µs** (61,5 ciclos a 15 MHz, medido en el firmware).
- **Margen: 46×.**

Simulado con el interruptor real: error entre el condensador de muestreo y el pin al cerrar la ventana = **0,0028 LSB** (canal V) y 0,0017 LSB (canal I), sobre 60 ventanas.

Ayuda mucho que `Cfv = 10 nF` sea 2000 veces el condensador de muestreo (5 pF): la redistribución de carga vale 5 p/10 n = 5e−4 de la diferencia entre muestras.

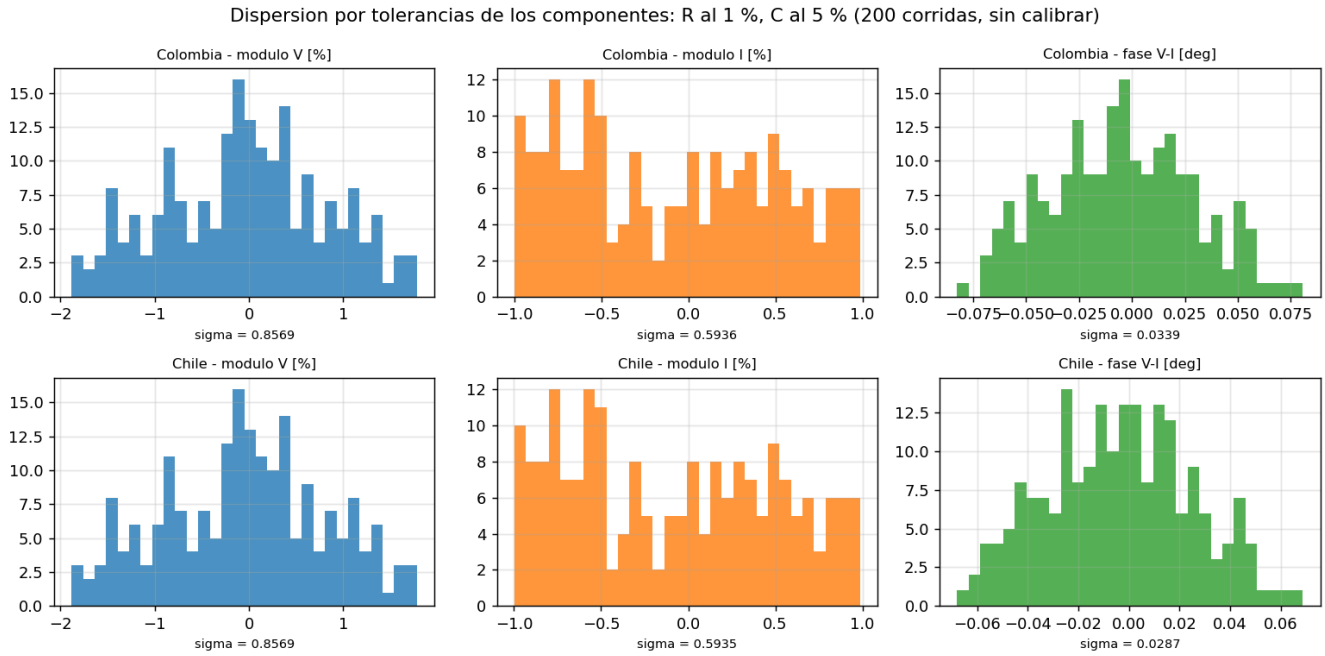
`CONFIRMAR`: la hoja del F303 no publica `R_ADC` ni `C_ADC`. Se usó R_ADC = 1 kΩ (del ejemplo de AN2834) y C_ADC = 5 pF (valor que ST reporta para F4/G4). Con 46× de margen, incluso equivocándose por un factor 10 se cumple.

5.5 Monte Carlo (200 corridas, R 1 %, C 5 %)

Error respecto al nominal, **sin calibrar**:

	módulo V [%]	módulo I [%]	fase V [°]	fase I [°]	fase V−I [°]
σ	0,857	0,594	0,024	0,025	0,034
p95 |·|	1,600	0,963	0,040	0,043	0,065
máx |·|	1,881	0,999	0,044	0,052	0,083

(Colombia; Chile sale igual en módulo y algo menor en fase. Las dos redes usan las mismas tiradas porque `mc()` de LTspice es determinista por índice de corrida: la comparación entre redes es pareada.)



El histograma del módulo I es plano porque lo domina una sola tirada uniforme (Rbi, que multiplica la ganancia directamente); el del módulo V es triangular porque se suman Rp y Rbv.

Coherente con la cadena de Python, que informa TVE (módulo **y** fase juntos) y además mueve la relación del CT y la magnetizante. Sin calibrar, el desfase del CT entra entero en el TVE (TVE ≈ fase en rad), así que:

Canal	Módulo (LTspice)	CT_RATIO	Fase del CT	Suma en cuadratura	TVE p95 de Python
V	1,60 %	—	0,689° = 1,20 %	2,00 %	**2,11 %**
I	0,96 %	0,95 %	0,833° = 1,45 %	1,97 %	**2,32 %**

Del orden correcto. No coincide al decimal porque el p95 de una suma no es la suma en cuadratura de los p95.

5.6 Cadena completa, calibrada a un punto

p95 sobre 200 combinaciones (`results_chain.txt`):

Escenario	Colombia TVE_V / TVE_I [%]	Chile TVE_V / TVE_I [%]
nominal	0,008 / 0,013	0,007 / 0,012
V 80 %, I 10 %	0,009 / 0,129	0,010 / 0,124
V 120 %, I 200 %	0,007 / 0,008	0,007 / 0,007
f0 − 2 Hz	0,059 / 0,071	0,083 / 0,102
f0 + 2 Hz	0,056 / 0,067	0,077 / 0,094
f0 − 5 Hz	**0,151 / 0,183**	**0,219 / 0,268**
f0 + 5 Hz	0,127 / 0,156	0,179 / 0,222

Error de fase V−I ≤ 0,068°; error de frecuencia ≤ 6,3 mHz.

---

6. Cambios de diseño propuestos (NO aplicados)

6.1 El burden del ZMPT101B es 4,7× la carga del ensayo de la hoja

La hoja especifica ≤20′ (0,333°) con **100 Ω**. El diseño usa **470 Ω**, y el error de fase escala con (Rw + Rb): sube a **0,69° a 60 Hz / 0,83° a 50 Hz**.

Residuo tras calibrar en f0, a f0 ± 5 Hz (Colombia / Chile):

Opción	ε_V a f0	Señal en el pin	Residuo fase V	Residuo **fase V−I**
A. Dejar 470 Ω (actual)	0,69° / 0,83°	0,91 V rms (1,29 V pico)	±0,063° / ±0,092°	**±0,013° / ±0,019°**
B. Bajar a 100 Ω	0,28° / 0,33°	0,19 V rms → **pierde 2,2 bits**	±0,025° / ±0,037°	**±0,051° / ±0,074°** ← peor
C. Dejar 470 Ω y compensar el polo del CT en el firmware	igual que A	igual que A	ver abajo	ver abajo

(Las filas A y B suponen `Lm` conocida exactamente: son el residuo *determinista*. Con C ese término se va a cero y lo que queda es la **incertidumbre** de `Lm`, ±30 %, que no es determinista y por eso se mide con el Monte Carlo de la cadena, no con esta tabla.)

Recomiendo C, y descarto B. B parecía la opción obvia (volver a la carga del ensayo de la hoja) pero es peor por dos motivos:

1. Pierde 2,2 bits de resolución, lo que multiplica por ~4,7 el TVE por ruido.
2. **Empeora la fase V−I.** Hoy ε_V = 0,69° y ε_I = 0,83° son parecidos, y al restar los canales se cancelan casi del todo. Bajar solo el burden de V desempareja los dos canales y el residuo V−I se multiplica por 4.

Es decir: los 470 Ω, que a primera vista parecían un error de diseño frente a la hoja de datos, resultan estar **bien elegidos** para la medida de potencia. Lo que falta no es cambiar el hardware, es compensar en software.

C es software puro.

C, en concreto: `Hv_nom` / `Hi_nom` ya multiplican por el polo del RC; basta multiplicar también por `jωLm/(jωLm + Rw + Rb)`. Ya está implementado en `simulate_chain.py` detrás de la bandera `CT_IN_NOMINAL` (`python codigo/simulate_chain.py --ct-nom`). Medido:

Escenario	Hoy	Con compensación	Mejora
Colombia f0 − 5 Hz, TVE_V / TVE_I	0,151 / 0,183 %	**0,042 / 0,051 %**	3,6×
Colombia f0 + 5 Hz	0,127 / 0,156 %	**0,035 / 0,045 %**	3,6×
Chile f0 − 5 Hz	0,219 / 0,268 %	**0,059 / 0,075 %**	3,7×
Chile f0 + 5 Hz	0,179 / 0,222 %	**0,048 / 0,064 %**	3,7×
fase V−I, Chile f0 − 5 Hz	0,068°	**0,051°**	1,3×
en f0	0,008 / 0,013 %	0,008 / 0,013 %	sin cambio

Cuesta dos multiplicaciones complejas por ventana (`Hv_nom` ya hace una por el RC).

Lo que queda después de compensar (0,035–0,075 % de TVE, ±0,05° de fase V−I) ya **no** es el error del CT: es la **incertidumbre de ±30 % en `Lm`**. Como es un error de pendiente y no de offset, una calibración en un solo punto no lo toca. Dos caminos para bajarlo, si alguna vez hiciera falta:

- **Medir `Lm` y `Rw`** de cada unidad (puntos CONFIRMAR 1 y 2): deja de ser incertidumbre y pasa a ser un número.
- **Calibrar en dos frecuencias** (p. ej. f0 y f0 + 5 Hz) y despejar `Lm` por canal y por unidad.

Ninguno es urgente: 0,075 % está trece veces por debajo del 1 % de la norma.

6.2 El margen contra 3,3 V queda en 106 mV en el caso peor

Con V al 120 % y I al 200 % simultáneos quedan 106 mV (3,2 % del fondo de escala). No está mal, pero no absorbe mucho. Opciones, si se quiere más margen: bajar Rbv de 470 a 430 Ω (−8,5 % de señal, +110 mV de margen) o Rbi de 30 a 27 Ω. **Ninguna aplicada**: hay que decidir si 106 mV alcanzan.

Ojo: los dos picos del caso peor no coinciden necesariamente en el tiempo (hay 30° de desfase V−I), así que 106 mV es el margen de cada canal por separado, que es lo que importa.

6.3 La pieza del BOM no coincide con el modelo

`params.py` y el README dicen **«SCT013-100»**, pero el modelo usa 2000:1 (100 A : 50 mA) con burden externo de 30 Ω, que es la **SCT-013-000**. En la nomenclatura de YHDC, la **SCT-013-100 es la de salida 100 A : 1 V, con burden interno**: si se compra esa, el burden de 30 Ω no va y toda la cadena del canal I cambia.

Hay que decidir cuál es la pieza real. Si es la -000, es solo renombrar en `params.py`, el README y los esquemáticos. No lo toqué porque cambia el BOM.

6.4 Rechazo de aliasing a 7,5 kHz

−17,3 dB (RC + CIC) en el Nyquist de la salida de 15 kS/s. Para armónicos de red no es problema (el 125.º armónico no existe), pero sí para ruido de conmutación. Es el punto más flojo de la cadena anti-aliasing. No propongo cambiarlo (subir el orden del filtro cuesta componentes y fase); lo dejo anotado.

---

7. Lo que se revisó y salió bien

- **Igualdad de los RC**: Rbv+Rfv = Rbi+Rfi = 3770 Ω exactos. La parte RC de la fase es idéntica en los dos canales. Lo que rompe la cancelación V−I no es el RC, son los transformadores.
- **Carga del ADC sobre el filtro**: despreciable. El pin ve 1 kΩ + 12 pF (7 pF de pad/PCB + ≈5 pF de los dos diodos): polo en 13 MHz, 0,0003° a 60 Hz.
- **Ruido del ADC**: la simulación supone 1,5 LSB rms. **Medido en la placa** (`STM32/uPMU/docs/RESULTADOS.md`, T3.4/T3.5): 0,62–0,66 LSB rms de ruido propio del ADC (VREFINT) y 1,07–1,40 LSB en una entrada externa con cable. El supuesto de la simulación es **conservador**: no hace falta cambiarlo.

- **Muestreo simultáneo V/I**: el firmware usa ADC1+ADC2 en modo dual simultáneo, **medido** con desfase < 0,012°. El AFE no añade error de tiempo entre canales.
- **Unidades y nombres de nodos**: coherentes entre `params.py`, los netlists y el esquemático. La única incoherencia encontrada es la del nombre de la pieza (§6.3).

Alimentación: lo ratiométrico no salva al canal

Conviene dejarlo escrito porque es contraintuitivo. El ADC entrega `código = 4096·V_pin/VDDA`. En el pin hay `VDDA/2 + s`, luego:

```
código = 2048 + 4096·s/VDDA
```

El **offset** sí es perfectamente ratiométrico (siempre 2048, pase lo que pase con VDDA, porque VCM sale de un divisor de VDDA). Pero la **señal** `s` viene de la red a través de Rp, no de VDDA: su código escala con **1/VDDA**. Es decir, el ruido y la deriva de VDDA entran como **ruido multiplicativo de ganancia**, y la medida ratiométrica no los cancela.

Cuánto: VREFINT medido da σ = 0,64 LSB sobre 1511 → 4,2e−4 relativo. Si todo fuera VDDA (no lo es: la mayor parte es ruido del propio ADC) serían 420 ppm rms de ruido de ganancia. Como es blanco y el estimador promedia 1000 muestras, queda en ~0,0013 % de TVE: despreciable. **Lo que sí importaría es una deriva lenta de VDDA** (térmica), que la calibración de un punto fija solo en el momento de calibrar. `CONFIRMAR`: medir VREFINT contra temperatura; el firmware ya lo lee y podría corregir VDDA en vivo.

---

8. Puntos marcados CONFIRMAR

#	Qué	Por qué importa	Cómo resolverlo
1	`RW_V = 150 Ω`, `RW_I = 20 Ω` (devanados secundarios)	Entran en ε = (Rw+Rb)/(ωLm): con Rw=0 el error de fase del ZMPT sube a 1,57° en vez de 0,83°	Óhmetro sobre las unidades reales
2	Frecuencia del ensayo de la hoja del ZMPT101B	Se supuso 50 Hz para derivar Lm; si fuera 60 Hz, Lm sale 20 % distinta	Hoja de datos completa del fabricante
3	Error de fase del SCT-013	No hay dato de YHDC. Se mantuvo el supuesto de 1° a 50 Hz	Medir contra una referencia, o pedir la hoja
4	`R_ADC = 1 kΩ`, `C_ADC = 5 pF` (STM32F303)	No están en la hoja del F303. De AN2834 y de valores de F4/G4	Tabla de características del ADC de la hoja correcta (DocID023353)
5	`R_PRI_ZMPT = 0`	Se suma a Rp; mueve la ganancia ~1 %	Óhmetro sobre el primario
6	Pieza real: SCT-013-000 vs SCT-013-100	Cambia todo el canal I (§6.3)	Mirar el rótulo
7	Saturación del núcleo	No modelada: no hay curva B-H. A 120 % de V y 200 % de I se está dentro de lo nominal (70 A de 100 A)	Ensayo con sobrecorriente, o curva del fabricante
8	Transitorio de encendido de ~3 s (§4.5)	Si el firmware reporta antes, esos fasores están mal	Medir en la placa con el front-end puesto
9	Máximos absolutos del pin del F303 con 3,88 V (§5.3)	Decide si el 1N4148 basta o hace falta un Schottky	Tabla de máximos absolutos
10	Deriva térmica de VDDA	No se cancela ratiométricamente (§7)	Medir VREFINT contra temperatura

Siguen vigentes los de la simulación original: el error de tiempo del PPS se evalúa aparte; no hay INL del ADC, ni armónicos, ni interferencia fuera de banda.

---

9. Archivos

```
uPMU_AFE/
├── run_ltspice.bat      punto de entrada (Windows): lo corre todo
├── codigo/              scripts de Python
├── netlists/            los 13 .cir y el esquemático uPMU_AFE.asc
├── figuras/             PNG de resultados y de las hojas del esquemático
├── resultados/          salidas de texto y JSON
└── docs/                este informe y las hojas del esquemático, en PDF
```

Archivo	Qué es
`codigo/params.py`	Fuente única de valores. Añadidos: modelo del CT (Lm, Rw), red del ADC, `R_PRI_ZMPT`
`codigo/rutas.py`	Dónde escribe cada script
`codigo/make_netlists.py`	Genera los 13 netlists
`codigo/make_asc.py`	Genera `netlists/uPMU_AFE.asc` y **verifica** que LTspice extraiga de él los mismos nodos que del netlist
`codigo/afe_model.py`	Modelo cerrado exacto de un canal. Autocomprobación: `python codigo/afe_model.py`
`codigo/ltspice_raw.py`	Lector de `.raw` ASCII. Autocomprobación: `python codigo/ltspice_raw.py`
`codigo/run_ltspice.py`	Regenera netlists y esquemático, corre LTspice en batch, analiza y grafica
`codigo/simulate_chain.py`	Cadena completa (ADC + CIC + estimador). Bandera `--ct-nom` para la propuesta 6.1-C
`codigo/draw_sheet1.py`, `draw_sheet2.py`, `draw_sheet3.py`	Las tres hojas del esquemático
`codigo/make_docs.py`	Markdown → PDF (Chrome/Edge headless)
`resultados/results_ltspice.txt`	AC, transitorio, clamps, muestreo del ADC y Monte Carlo
`resultados/results_chain.txt`, `results_chain_ctnom.txt`	Cadena completa, con el firmware de hoy y con la compensación propuesta
`figuras/ltspice_*.png`	Las tres gráficas de resultados
`docs/uPMU_AFE_hoja3_modelo_ltspice.pdf`	**El circuito simulado, dibujado.** La hoja 1 es la de montaje

El esquemático para abrir en LTspice

`netlists/uPMU_AFE.asc` se abre con File → Open en LTspice y se corre con Run: es el circuito dibujado, no el netlist de texto. Lo genera `make_asc.py` desde `params.py`, igual que los `.cir`, y **no se da por bueno sin verificarlo**: se le pide a LTspice que extraiga el netlist del esquemático (`-netlist`) y se comprueba que los 28 componentes queden con los mismos nodos que `uPMU_AFE_tran.cir`. Simulado, da el mismo resultado punto a punto (0,0000 mV de diferencia).

Netlists: `uPMU_AFE_ac_{V,I}.cir` (AC por canal), `uPMU_AFE_ac_ideal_{V,I}.cir` (CT ideal, para el antes/después), `uPMU_AFE_tran[_Chile][_ovl].cir`, `uPMU_AFE_clamp.cir`, `uPMU_AFE_adcsh.cir`, `uPMU_AFE_mc_{Colombia,Chile}.cir` (Monte Carlo) y `uPMU_AFE_LTspice.cir` (para abrir a mano en la GUI).